



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

SOLUCIÓN TÉCNICA Y ACADÉMICA -
LIDERAZGO, EQUIPOS Y CULTURA DE INGENIERÍA

Profesor:
Márquez Félix

Bachiller:
Alondra Moreno CI: 30.331.870

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Puerto Ordaz septiembre de 2026

PREGUNTA 1: LA MANIOBRA INVERSA DE CONWAY Y TEAM TOPOLOGIES (CASO BANCO MONOLÍTICO)

1. Análisis del Problema según la Ley de Conway

La Ley de Conway (Melvin Conway, 1967) establece que: 'Las organizaciones que diseñan sistemas están constreñidas a producir diseños que son copias de las estructuras de comunicación de estas organizaciones'.

En el caso del banco tradicional, la estructura organizacional actual está dividida en silos funcionales estrictos: un Departamento de Base de Datos, un Departamento de Backend Java y un Departamento de Interfaces de Usuario (UI), coordinados horizontalmente por un Gerente de Proyecto tradicional.

Razones Fundamentales del Fracaso de la Agilidad y los Meses de Retraso:

- **1. Handoffs Secuenciales y Tiempos de Espera (Queue Time):** La estructura social (los silos) refleja exactamente la arquitectura del código (el monolito). Para entregar una sola funcionalidad del negocio (ej. 'Aprobación de un Crédito'), se requiere una transferencia secuencial de mano en mano (hand-off) entre departamentos: la solicitud viaja del equipo UI al equipo Backend, luego al equipo DB, y regresa de vuelta. Esta cadena de comunicación indirecta genera altísima fricción, tiempos de espera masivos y bloqueos interdepartamentales.
- **2. Gestión de Dependencias Burocrática:** El Gerente de Proyecto intenta gestionar las dependencias cruzadas mediante reuniones y cronogramas en cascada. Sin embargo, como cada departamento tiene sus propias prioridades y jefaturas, las dependencias no se resuelven en tiempo real, acumulando meses de demora para cambios triviales.
- **3. La Trampa de la 'Falsa Agilidad' (Agile in Name Only):** Aunque la gerencia declare que adopta 'Scrum', la Ley de Conway garantiza que si la estructura de comunicación sigue siendo monolítica y fragmentada por capas tecnológicas, el sistema resultante seguirá siendo un Monolito Altamente Acoplado e Imposible de Desplegar de forma independiente.

2. Rediseño Organizacional con Team Topologies (Skelton & Pais, 2019)

Para romper el monolito arquitectónico, debemos aplicar la Maniobra Inversa de Conway (Inverse Conway Maneuver): rediseñar primero la estructura de los equipos y sus líneas de comunicación para que la arquitectura de software deseada (microservicios y Contextos Delimitados / Bounded Contexts) emerja de forma natural.

Basándonos en el marco de Team Topologies (Skelton & Pais, 2019), reorganizamos la organización en 4 tipos de equipos especializados:

- **1. Stream-Aligned Team (Equipo Alineado al Flujo - Dominio Créditos):** Equipo multidisciplinario e independiente (Frontend, Backend, QA) alineado de extremo a extremo a un flujo de valor de negocio específico ('Dominio de Créditos'). Es propietario del ciclo de vida completo de los microservicios de Créditos (desde la idea hasta producción), reduciendo las dependencias a cero.
- **2. Platform Team (Equipo de Plataforma):** Equipo de ingeniería que construye y mantiene una 'Plataforma Interna de Autoservicio' (Internal Developer Platform). Proporciona a los Stream-aligned teams infraestructura como código, pipelines de CI/CD preconfigurados, entornos de prueba y aprovisionamiento de bases de datos mediante APIs simples, liberando a los desarrolladores de la carga cognitiva infraestructural.
- **3. Enabling Team (Equipo Facilitador / Capacitador):** Equipo de especialistas multifuncionales (expertos en microservicios, seguridad o arquitectura ágil) cuya función no es programar features, sino capacitar y mentorear a los Stream-aligned teams durante periodos cortos (ej. 2 a 4 semanas) para cerrar brechas técnicas y transferir habilidades.
- **4. Complicated-Subsystem Team (Equipo de Subsistema Complicado):** Equipo altamente especializado dedicado exclusivamente a desarrollar un subsistema matemáticamente complejo o de alta densidad técnica (ej. el 'Motor de Cálculo de Riesgo Crediticio' o el 'Core de Algoritmos Financieros'). Encapsula esa complejidad detrás de un contrato de API claro para que el equipo de Créditos lo consuma sin sobrecargarse.

Soporte a Microservicios y Contextos Delimitados (Bounded Contexts):

Esta reorganización alinea directamente a los Stream-Aligned Teams con los Contextos Delimitados (Bounded Contexts) de Domain-Driven Design (DDD). Cada equipo es dueño absoluto de su dominio de datos y servicios, eliminando los silos funcionales de DB/Backend/UI y permitiendo despliegues independientes en producción.

3. El Nuevo Rol del Antiguo 'Gerente de Proyecto'

En una estructura descentralizada orientada a flujos de valor y productos, el rol tradicional del Gerente de Proyecto (basado en asignación de tareas, seguimiento de cronogramas y control top-down) se vuelve obsoleto y contraproducente.

Evolución del Rol hacia el Liderazgo Servicial (Servant Leadership):

- **1. Gestor del Flujo de Valor (Value Stream Orchestrator):** Pasa a actuar como Scrum Master, Agile Coach o Value Stream Manager, enfocando su trabajo en optimizar el flujo de trabajo (Flow Metrics: Lead Time, Cycle Time, Throughput) y eliminar cuellos de botella organizacionales.
- **2. Liderazgo Servicial y Remoción de Impedimentos:** En lugar de 'mandar y controlar', el líder sirve al equipo removiendo impedimentos externos, protegiéndolos de interrupciones políticas y facilitando la colaboración inter-equipos.
- **3. Facilitador de Autonomía:** Fomenta la autonomía responsable y la toma de decisiones descentralizada en los equipos alineados al flujo, migrando de supervisar tareas a medir resultados de negocio (OKRs).

PREGUNTA 2: LIDERAZGO, CONFLICTO Y SEGURIDAD PSICOLÓGICA (CASO DE ESTUDIO TÉCNICO)

1. Diagnóstico de Seguridad Psicológica

La Seguridad Psicológica (Amy Edmondson, 1999) se define como: 'La creencia compartida por los miembros de un equipo de que el entorno es seguro para tomar riesgos interpersonales sin temor a ser humillado, castigado o ignorado'. En el emblemático Proyecto Aristotle de Google, se identificó que la seguridad psicológica es el factor número uno determinante del alto rendimiento en equipos de ingeniería.

Diagnóstico de la Situación del Equipo:

El comportamiento del Arquitecto Principal (comentarios sarcásticos, pasivo-agresivos y destructivos en las revisiones de Pull Requests hacia desarrolladores junior) representa una violación directa a la seguridad psicológica del equipo.

Impacto Catastrófico a Largo Plazo:

- **1. Parálisis de la Innovación y Miedo al Riesgo:** Los desarrolladores junior y mediamente experimentados dejan de proponer ideas innovadoras o soluciones alternativas por miedo al escarnio público en GitHub/GitLab. La innovación se reduce a cero.
- **2. Retrospectivas Estériles y Ocultamiento de Errores:** Las reuniones de retrospectiva se convierten en eventos silenciosos e inútiles donde nadie menciona los errores reales ni la deuda técnica por temor a represalias o críticas del Arquitecto.
- **3. Degradación de la Velocidad y Rotación de Personal:** Para evitar revisiones dolorosas, los desarrolladores envían PRs gigantescos a última hora o intentan ocultar fallos, lo que ralentiza el flujo de entregas, degrada la calidad del código y dispara la rotación de personal (renuncia de talentos).

2. Estrategia de Intervención (Liderazgo Servicial y Comunicación No Violenta - CNV)

El Agile Coach / Líder Técnico debe abordar el conflicto utilizando el marco de Comunicación No Violenta (CNV de Marshall Rosenberg), estructurado en 4 pasos objetivos:

Parte A: Sesión Privada 1-a-1 con el Arquitecto Principal

- **1. Observación Objetiva (sin juicios de valor):** 'He notado que en los últimos 5 Pull Requests has utilizado comentarios como "esto es código de principiante" o "no entiendes nada de arquitectura".'
- **2. Expresión del Impacto/Sentimiento:** 'Me preocupa que el equipo junior se sienta intimidado y retraído al recibir retroalimentación con ese tono.'
- **3. Necesidad Organizacional:** 'Como equipo de ingeniería, necesitamos mantener altos estándares técnicos, pero también garantizar un entorno de aprendizaje seguro donde los juniors puedan crecer sin temor.'
- **4. Petición Concreta y Accionable:** 'Te pido que enfoques la retroalimentación exclusivamente en la mejora del código utilizando sugerencias constructivas y que me ayudes a definir una guía objetiva de revisión de PRs.'

Parte B: Reestructuración Técnica del Proceso de Code Review (Objetivación del Feedback)

Para eliminar la subjetividad y los ataques personales en las revisiones de código, se implementan tres mecanismos técnicos:

- **1. Automatización Estática (Quality Gates):** Se delega la revisión de estilo, formateo y reglas básicas a herramientas automatizadas de CI (ESLint, Prettier, SonarQube). El Arquitecto ya no discute sobre comas o sintaxis; la máquina lo bloquea automáticamente.
- **2. Plantillas Estándar de PR:** Se establece una plantilla obligatoria de PR con una rúbrica de revisión clara (Seguridad, Rendimiento, Legibilidad).
- **3. Guía de Etiqueta en Code Reviews:** Se adopta una convención de comentarios respetuosa (ej. Conventional Comments: `suggestion:`, `question:`, `issue:`). Los comentarios deben ser redactados en forma de propuesta constructiva ('¿Qué opinas si aplicamos el patrón Strategy aquí para facilitar las pruebas?') en lugar de críticas ad hominem.

3. Dinámica para la Retrospectiva: 'El Verificador de Seguridad y el Lienzo de Confianza'

Propuesta práctica para la próxima Sprint Retrospective enfocada en restaurar la seguridad psicológica y rescatar la voz del equipo:

Fase 1: Safety Check (Verificación de Seguridad Anónima)

Antes de iniciar la reunión, cada miembro vota de forma 100% anónima en una escala del 1 al 5 respondiendo: '¿Qué tan seguro te sientes de hablar de problemas reales en este equipo sin temor a consecuencias negativas?'. Si el promedio es menor a 3.5, la sesión se enfoca exclusivamente en hablar de dinámicas de equipo y respeto interpersonal, congelando los temas técnicos.

Fase 2: Dinámica 'Elogios y Oportunidades de Aprendizaje' (Appreciation & Learning)

- **1. Tablero Silencioso:** Se utiliza un tablero virtual (Miro/Mural) dividido en dos columnas: 'Prácticas que nos ayudan a crecer' y 'Comportamientos que debemos detener'.
- **2. Escritura Equitativa:** Durante 10 minutos, todos escriben notas adhesivas de forma silenciosa (escritura anónima previa), garantizando que los desarrolladores junior tengan exactamente el mismo peso visual y espacio que el Arquitecto Principal.
- **3. Conversación Facilitada:** El facilitador (Agile Coach) lee las notas sin atribuir autores y promueve acuerdos de equipo (Team Agreements), como: 'Todos los errores en el código son responsabilidad del sistema y del proceso de pruebas, no de la persona'.